CS4071 – Project Report – Recycle Rush

24405094 – Ciaran Fernandez Walshe

1. Introduction:

For my project I decided to develop a virtual reality recycling game. The player takes on the role of a worker in a recycling facility, where they must sort different types of waste into the appropriate recycling bins. The objective is to recycle as much waste as possible to get a high score before the time runs out.

The game features a conveyor belt that continuously transports rubbish towards the player. The player uses VR controllers to pick up individual items and throw them into the correct recycling bins behind the conveyor. The waste is divided into four categories: paper, plastic, metal and other waste. Correctly sorting items increases players score, while incorrect sorting decreases the players' score. Items that do not belong in a recycling bin must be left on the conveyor to reach the incinerator at the end.

Virtual reality was chosen as I was interested to learn more about it and how to develop for it. The project also explores other aspects of game development such as physics-based object interaction, collision detection, object spawning, user interface design.

2. Resources:

I used Unity 6.3 LTS as my game engine for the project. To learn the essentials of virtual reality development, I began with the Unity Learn VR pathway, which introduced the fundamental concepts of VR interaction and setting up a project for a VR headset. I then used Valem's VR development tutorial series on YouTube to expand my understanding of VR development.

We learned the basics of Unity and C# scripting in our "Virtual and Augmented Reality Design" and "Artificial Intelligence for Games" modules.

To improve the visual quality of the environment, I used 3D models and textures sourced from the Unity Asset Store. Music was sourced from itch.io, while sound effects were obtained from Pixabay to add audio feedback and atmosphere to the project.

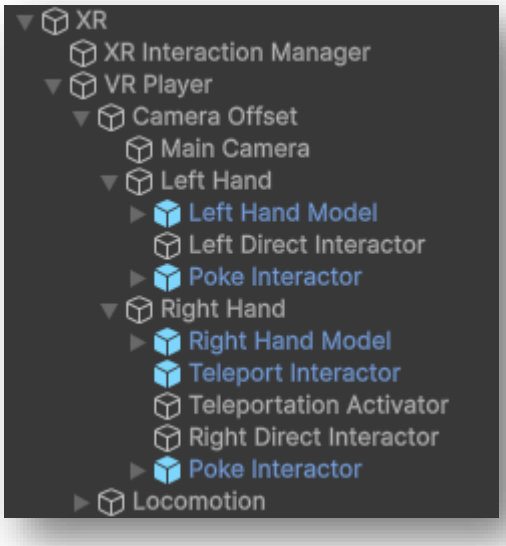
3. Process:

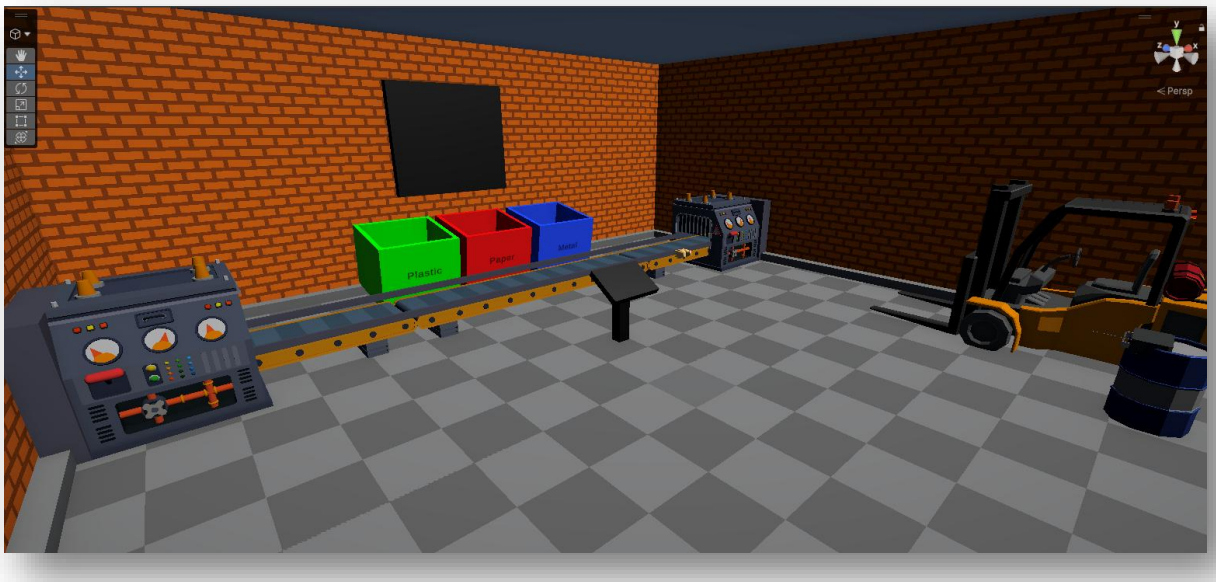
3.1 Initial Setup, VR Interaction and Environment:

I began development by following the tutorials I mentioned above. These resources helped me understand how to set up a VR project and implement interactions using the XR Interaction Toolkit such as VR locomotion, head tracking, teleportation, controller input, grabbable objects, sockets and VR UI.

I created the initial Unity project, configured the XR Origin and set up the controller interactions. I used the XR Device Simulator to test the project because I did not have access to a VR headset during the initial development.

Once the basic interactions were working, I began constructing the recycling facility. I used models from the Unity Asset Store to create the room, machines, objects and decoration I would need. I arranged the room around the conveyor belt and recycling bins as they were the central focus.



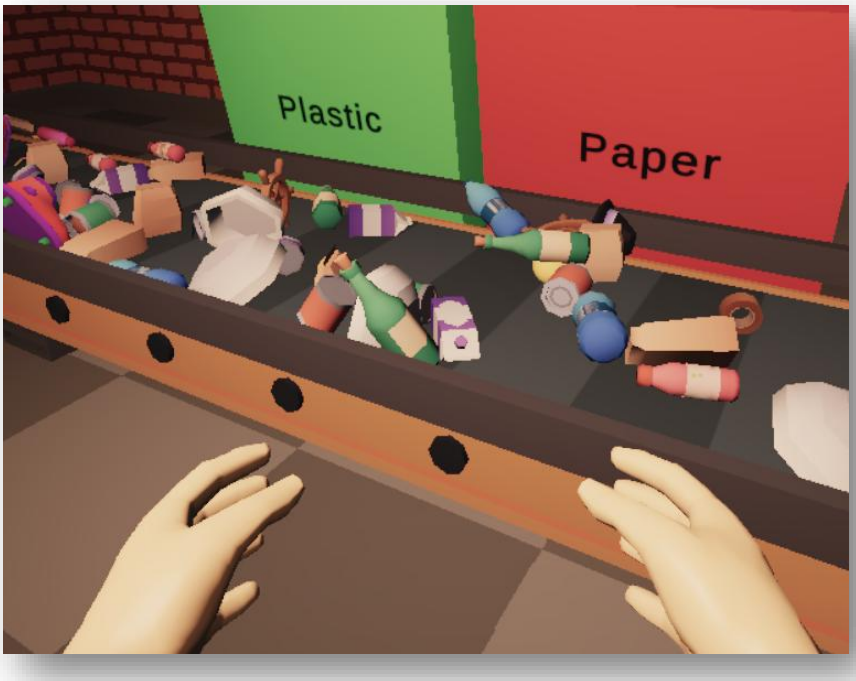


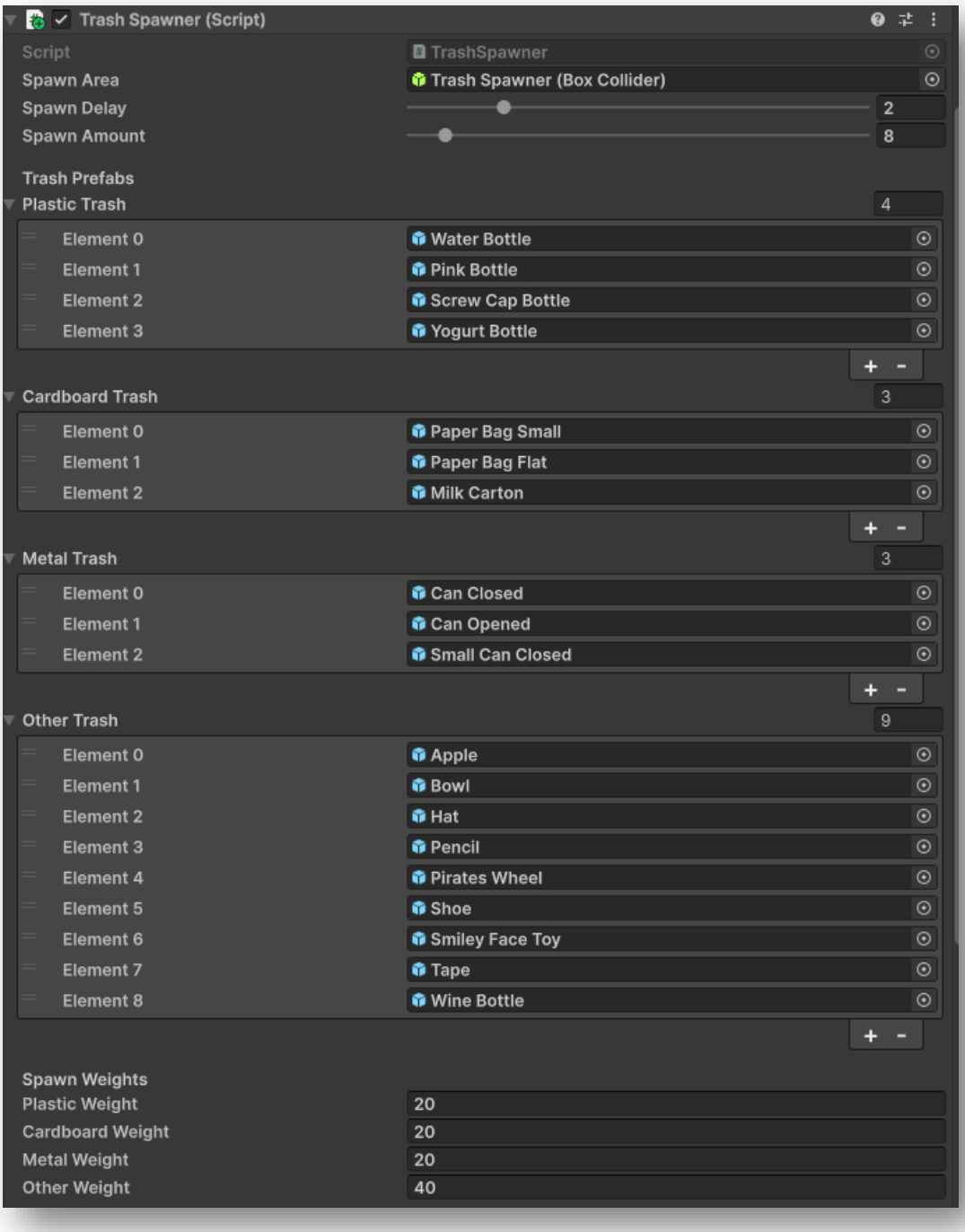
3.2 Conveyor Belt and Rubbish Spawning:

After creating the room, I implemented the rubbish spawner. I created a collection of rubbish prefabs and configured the spawner to select rubbish using weighted random selection. This allowed me to control how frequently different types of rubbish appeared. I also added a delay between spawns to control how much rubbish there would be on the conveyor.

The conveyor belt was one of the most challenging features to implement. My initial approach was to apply a force to objects while they were on the belt. However, this made the rubbish move unpredictably or fly off the belt. I then tried freezing their physics while they were on the conveyor and re-enabling it when the player grabbed them. Although this made the system easier to control it made the game less realistic and immersive.

In the end I returned to the original physics-based approach and modified it to reduce sideways movement and unwanted rotation by adding counter forces. I also added rails along the sides of the conveyor to allow the rubbish to move around more freely on the belt without being able to fall off the edge. This was the best I could achieve and even though the result was not perfect I left it as is to move on to other parts of the game.



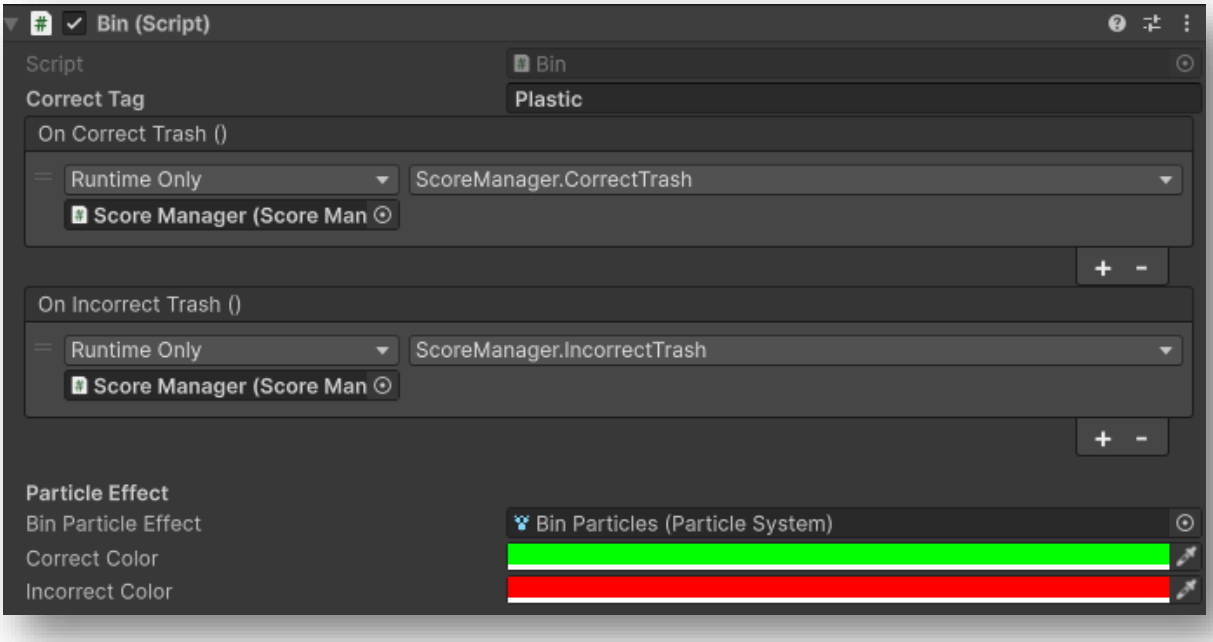

Trash Spawner (Script)
Script
TrashSpawner
Spawn Area
Trash Spawner (Box Collider)
Spawn Delay
2
Spawn Amount
8
Trash Prefabs
Plastic Trash
4
Element 0
Water Bottle
Element 1
Pink Bottle
Element 2
Screw Cap Bottle
Element 3
Yogurt Bottle
Cardboard Trash
3
Element 0
Paper Bag Small
Element 1
Paper Bag Flat
Element 2
Milk Carton
Metal Trash
3
Element 0
Can Closed
Element 1
Can Opened
Element 2
Small Can Closed
Other Trash
9
Element 0
Apple
Element 1
Bowl
Element 2
Hat
Element 3
Pencil
Element 4
Pirates Wheel
Element 5
Shoe
Element 6
Smiley Face Toy
Element 7
Tape
Element 8
Wine Bottle
Spawn Weights
Plastic Weight
20
Cardboard Weight
20
Metal Weight
20
Other Weight
40

3.3 Recycling Bins, Incinerator and Scoring System:

Once the conveyor was finished, I created the recycling bins and implemented their logic. Each bin was configured to accept a specific waste category. I used tags to identify the rubbish types and trigger colliders to detect when objects entered a bin. When an item entered the trigger, the script compared its tag against the bin's accepted category to decide whether it had been sorted correctly.

I then created a score manager script to handle the scoring. Correctly sorted items increased the player's score, while incorrect items resulted in a penalty. I kept the scoring logic in a separate script to allow me to connect it to the user interface easily later.

I also created an incinerator at the end of the conveyor. It had a simple trigger collider that would destroy the trash game objects once they were inside.



3.4 User Interface, Tablet and Game Loop:

After completing the main gameplay mechanics, I developed the user interface. I created a tablet in the room and added an interactive start button so that the player could begin a round by pressing it with their VR hands using a poke interactor. I connected the tablet to the game's management scripts and implemented the score display and countdown timer.

I then connected these systems into a complete game loop. Pressing the start button resets the score and timer, activates the rubbish spawner and begins the round. When the countdown reaches zero, the game stops spawning rubbish, removes the remaining objects and displays the final score. I also added persistent best-score tracking so that the player's highest score is saved between game sessions.

```
using UnityEngine;

0 references | Unity Script (1 asset reference)
public class GameManager : MonoBehaviour
{
    4 references | Unity Serialized Field = Trash Spawner
    [SerializeField] private TrashSpawner trashSpawner;
    3 references | Unity Serialized Field = Score Manager
    [SerializeField] private ScoreManager scoreManager;
    2 references | Unity Serialized Field = Game Timer
    [SerializeField] private GameTimer gameTimer;

    2 references | Unity Serialized Field = Back Wall Screen
    [SerializeField] private ScreenUI screenUI;
    2 references | Unity Serialized Field = Tablet
    [SerializeField] private TabletUI tabletUI;

    0 references
    public void StartGame()
    {
        ResetGame();

        trashSpawner.StartSpawning();
        gameTimer.StartTimer();

        screenUI.ShowGameScreen();
        tabletUI.ShowStopScreen();
    }

    1 reference
    public void ResetGame()
    {
        scoreManager.ResetScore();
        trashSpawner.ResetSpawner();
        gameTimer.ResetTimer();
    }

    0 references
    public void GameOver()
    {
        trashSpawner.ResetSpawner();
        trashSpawner.RemoveAllTrash();

        scoreManager.CheckBestScore();

        screenUI.ShowGameOverScreen(scoreManager.GetScore());
        tabletUI.ShowStartScreen();
    }
}
```

```
using UnityEngine;
using TMPro;

3 references | Unity Script (1 asset reference)
public class ScreenUI : MonoBehaviour
{
    2 references | Unity Serialized Field = Game Screen
    [SerializeField] private GameObject gameScreen;
    2 references | Unity Serialized Field = Game Over Screen
    [SerializeField] private GameObject gameOverScreen;

    1 reference | Unity Serialized Field = Score Text
    [SerializeField] private TMP_Text scoreText;
    1 reference | Unity Serialized Field = Final Score Text
    [SerializeField] private TMP_Text finalScoreText;
    1 reference | Unity Serialized Field = Time Remaining Text
    [SerializeField] private TMP_Text timeText;
    1 reference | Unity Serialized Field = [2 items]
    [SerializeField] private TMP_Text[] bestScoreTexts;

    2 references
    public void UpdateBestScoreTexts(int bestScore)
    {
        foreach (TMP_Text bestScoreText in bestScoreTexts)
        {
            bestScoreText.text = "Best Score: " + bestScore;
        }
    }

    1 reference
    public void UpdateScoreText(int score)
    {
        scoreText.text = "Score: " + score;
    }

    1 reference
    public void UpdateTimeText(float timeRemaining)
    {
        int minutes = Mathf.FloorToInt(timeRemaining / 60f);
        int seconds = Mathf.FloorToInt(timeRemaining % 60f);

        timeText.text = string.Format("{0:00}:{1:00}", minutes, seconds);
    }

    1 reference
    public void ShowGameScreen()
    {
        gameOverScreen.SetActive(false);
        gameScreen.SetActive(true);
    }

    1 reference
    public void ShowGameOverScreen(int finalScore)
    {
        gameScreen.SetActive(false);
        gameOverScreen.SetActive(true);

        finalScoreText.text = finalScore.ToString();
    }
}
```

3.5 Audio, Tutorial, Lighting and Final Testing:

Once the game loop was complete, I added audio to improve the atmosphere of the game. I added a radio to the scene, using a socket the player could place it in its place in the room, and it would snap to its correct position. I then added some sound effects to the machinery and sounds for when the player correctly or incorrectly sorted rubbish.

I created a tutorial screen to explain the controls and introduce the different waste categories. The tutorial was divided into separate pages, with navigation buttons allowing the player to move between the instructions before starting the game. It was also interactable the same way as the tablet. I used a script to capture photos of all my rubbish prefabs and create icons from them for the tutorial.

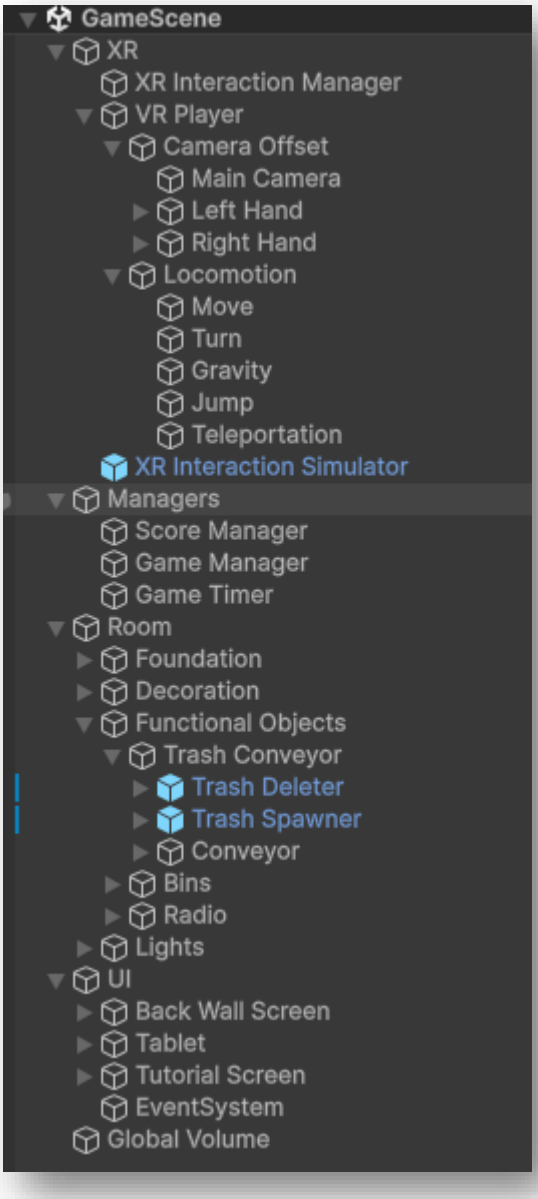
I also added lighting and post-processing to the room to make it more atmospheric. I placed point lights on the ceiling to light the room and placed a spotlight facing onto the conveyor and recycling bins to highlight them as important objects. I then added a global volume to the scene to add some effects such as tone mapping, colour adjustments, bloom and white balance.

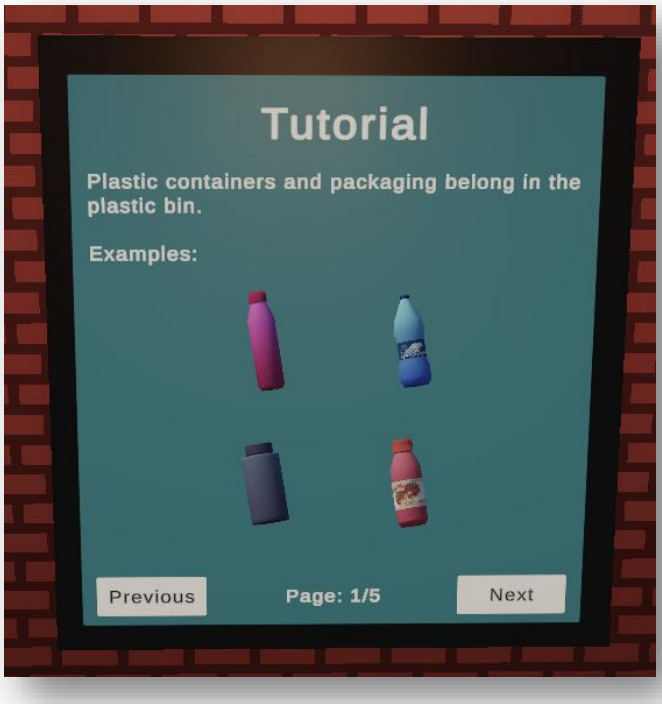
Another thing I added was particle effects for when rubbish is thrown into recycling bins.

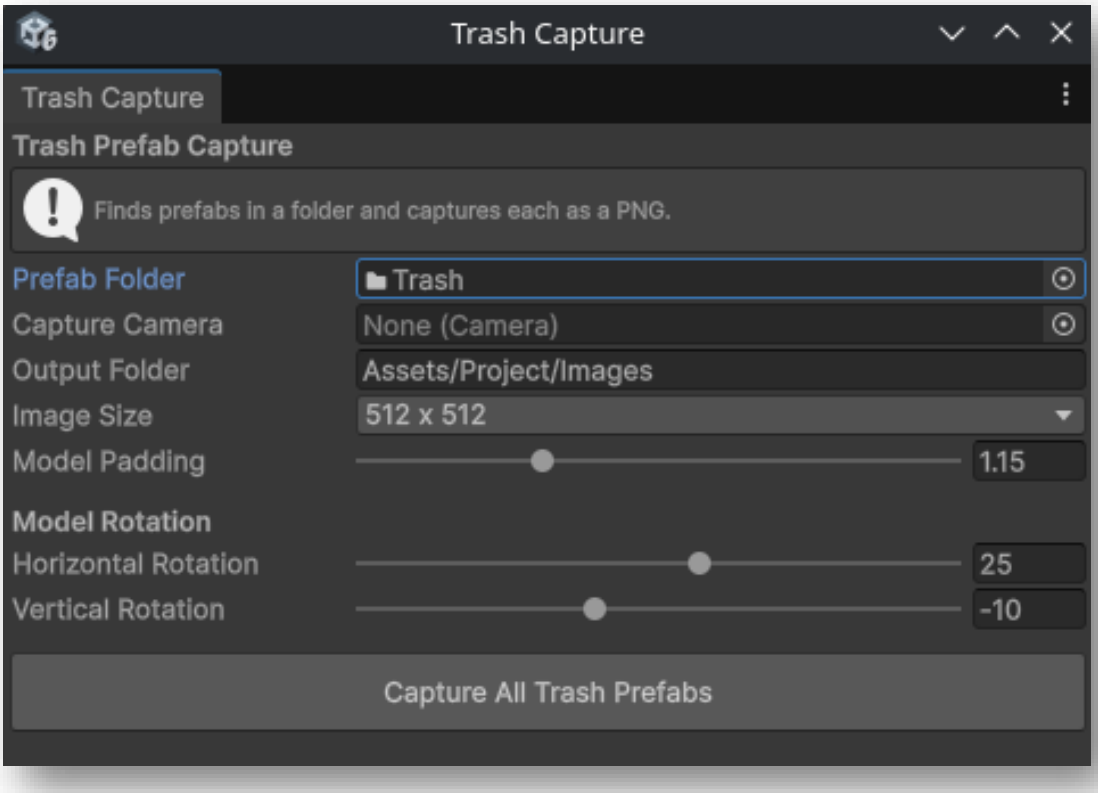
Finally, I borrowed a VR headset from the university to test the project on real hardware. Until this point, I had relied on the XR Device Simulator for testing. Testing on the headset allowed me to check the actual controller interactions, teleportation, object grabbing and user interface positioning. I made final adjustments based on these tests like fixing some VR specific collision bugs and adjusting the attach points, mass and scale of the different kinds of rubbish.

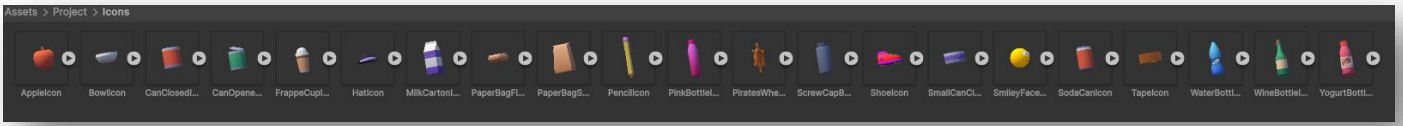
Once I was happy with the game, I made a final build to record the video for the submission.




GameScene
XR
XR Interaction Manager
VR Player
Camera Offset
Main Camera
Left Hand
Right Hand
Locomotion
Move
Turn
Gravity
Jump
Teleportation
XR Interaction Simulator
Managers
Score Manager
Game Manager
Game Timer
Room
Foundation
Decoration
Functional Objects
Trash Conveyor
Trash Deleter
Trash Spawner
Conveyor
Bins
Radio
Lights
UI
Back Wall Screen
Tablet
Tutorial Screen
EventSystem
Global Volume


Tutorial
Plastic containers and packaging belong in the plastic bin.
Examples:
Previous
Page: 1/5
Next


Trash Capture
Trash Capture
Trash Prefab Capture
Finds prefabs in a folder and captures each as a PNG.
Prefab Folder
Trash
Capture Camera
None (Camera)
Output Folder
Assets/Project/Images
Image Size
512 x 512
Model Padding
1.15
Model Rotation
Horizontal Rotation
25
Vertical Rotation
-10
Capture All Trash Prefabs


Assets > Project > Icons

4. Reflection:

As this was one of my first modules using Unity, the project provided an opportunity to learn many of the fundamentals of game development. While working on the project, I developed a better understanding of the Unity editor, scene management, GameObjects, components, physics and collision detection, developing a VR environment and the overall game development process. I really enjoyed seeing how everything I worked on came together to create a complete game.

Even though the module was not focused on programming, I spent a good bit of my time creating C# scripts. As I particularly enjoy the programming side of game development. It was quite a challenge going from the more traditional Java programming that we had been doing up until this semester to the much different way Unity handles C# scripting. However, it was also very rewarding to learn and get to know. I really enjoyed working on the script that would iteratively create icons of all the rubbish prefabs. It was cool to see how you could make your own custom tools withing Unity.

If I had more time, I would focus on improving the visual and audio quality of the game, I would add more sound effects to provide better feedback to the player and look for better quality models or attempt to create them myself. For example, I could not find a good recycling bin model, so I constructed one using basic shaped. I would also add a way for the user to adjust settings such as VR locomotion options and audio volumes.

Overall, I really enjoyed working on this project. Developing the project from an initial scene into a playable game was very rewarding, and then testing my game on an actual headset was very cool. The project has given me a strong foundation in Unity and has increased my interest in exploring VR development further potentially for my final year project.

5. Appendix:

Learning Resources:

- Unity Learn, VR Development Pathway
- Valem, VR Development Tutorial Series
- UI Toolkit, Custom Tooling
- Unity Documentation

3D Assets:

- Low Poly Factory Machine Pack (Demo)
- Low Poly Industrial Pack
- Low Poly Forklift
- Cute Supermarket Lite
- Low Poly Objects Pack
- Modern LED Lighting Pack

Audio:

- Voltz Supreme, Analog Ambience
- Freesound Community, SFX

Inspiration:

- Sushi Ben, Big Brane Studios Inc.
- Demeo, Resolution Games
- Rec Room, Thunderstorm Games
- Tiny Flock, No Brakes Games